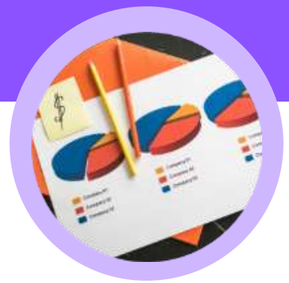


My Data Scientist Workflow Simulation

Customer Churn Analysis

Task

In this case study, I will analyze **customer behavior** using the existing database to determine which customers are **likely to churn** (i.e., stop transacting and leave the company).

Knowing this allows the company to **intervene proactively** based on the generated predictions. If a customer's behavior suggests they are **at risk of churning**, the company can take steps to **mitigate this possibility** and prevent the loss.

Task

My process will involve three key stages:

- **Data Retrieval:** I will extract the necessary data from the database using SQL queries.
- **Data Visualization:** I will then visualize the data and derive initial insights using Power BI.
- **Predictive Modeling:** Finally, i will implement a machine learning prediction model to accurately identify the customers who are expected to churn.

Workflow

01

Data Retrieval

02

Data Visualization

03

Predictive Machine Learning Model

Data Retrieval

I will begin by extracting the necessary data from a database. For this project, I will be utilizing the free Customer Orders (CO) database, which is accessible via freesql.com.
I require several key features (columns) to be pulled directly from the database:

- Customer_id
- Order_id
- Store_id
- Product_id
- Shipment_id
- Order_status
- Quantity

1

Data Retrieval

In addition to these raw features, i need to perform some aggregation and data transformation to calculate derived metrics crucial for our analysis:

1. Revenue: Calculated by multiplying Quantity by Unit_price.

Formula: **Revenue** = Quantity * Unit_price

2. Days_elapsed_tmax (Time Since the Latest Order in the Entire Dataset):

- **What it measures:** This is the duration, in days, between the date of a specific order and the date of the most recent order across the entire database. This helps measure the "freshness" of the entire dataset.

  Calculation: Max(Order_date) - Order_date (where Max(Order_date) is the latest order date in the whole table).

3. Days_from_cust_last_order (Recency of the Customer's Previous Order):

- **What it measures**: This is the duration, in days, between the date of the current order and the customer's previous order date. This metric is crucial for calculating the average time between a customer's orders.
- **Calculation**: Max(Order_date) OVER (PARTITION BY Customer_id) - Order_date
- **Simplified Concept:** For a specific customer, find their latest order date (the Max(Order_date) per Customer_id) and subtract the date of the current order.

Data Retrieval

[SQL Worksheet]*

```sql
SELECT
    O.CUSTOMER_ID AS CUST_ID,
    O.ORDER_ID AS ORDER_ID,
    TO_CHAR(O.ORDER_TMS, 'YYYY-MM-DD HH24:MI:SS')
    AS ORDER_DATETIME,
    TRUNC(
        MAX(O.ORDER_TMS) OVER ()
    ) - TRUNC(O.ORDER_TMS) AS DAYS_ELAPSED_TMAX,
    TRUNC(
        MAX(O.ORDER_TMS) OVER (PARTITION BY O.CUSTOMER_ID)
    ) - TRUNC(O.ORDER_TMS) AS DAYS_FROM_CUST_LAST_ORDER,
    OI.UNIT_PRICE * OI.QUANTITY AS REVENUE,
    OI.QUANTITY,
    O.STORE_ID,
    OI.PRODUCT_ID,
    O.ORDER_STATUS,
    S.SHIPMENT_STATUS
FROM
    CO.ORDERS O
JOIN
    CO.ORDER_ITEMS OI ON O.ORDER_ID = OI.ORDER_ID
LEFT JOIN
    CO.SHIPMENTS S ON OI.SHIPMENT_ID = S.SHIPMENT_ID
ORDER BY
    O.CUSTOMER_ID,
    O.ORDER_TMS;
```

Data Retrieval

erties | Tools | External Table Data | Table Style Options

COMPLETE

	B	C	D	E	F	G	
ID	ORDER_ID	ORDER_DATETIME	DAYS_ELAPSED_TMAX	DAYS_FROM_CUST_LAST_ORDER	REVENUE	QUANTITY	STOR
3	1	04/02/2021 13.20	432	350	148	4	
3	1	04/02/2021 13.20	432	350	6138	2	
45	2	08/02/2021 20.58	428	374	2598	3	
45	2	08/02/2021 20.58	428	374	2825	5	
18	3	09/02/2021 23.17	427	366	433	5	
45	4	10/02/2021 13.43	426	372	9192	4	
45	4	10/02/2021 13.43	426	372	15664	4	
45	4	10/02/2021 13.43	426	372	5642	2	
2	5	11/02/2021 18.01	425	210	13624	4	
2	5	11/02/2021 18.01	425	210	1695	3	
74	6	13/02/2021 20.11	423	365	11484	3	
74	6	13/02/2021 20.11	423	365	8464	4	
9	7	22/02/2021 00.57	414	291	19648	4	
9	7	22/02/2021 00.57	414	291	2471	1	

Simple Data Interpretation

Here is an example of the resulting data, sorted from the oldest transaction to the newest.
We need to understand two key numbers from the first record (Order_id 1):

Days_elapsed_tmax: 432

Simple Meaning: This order happened 432 days ago when compared to the latest date in our entire database.

Days_from_cust_last_order: 350

Simple Meaning: This specific customer placed their very last order 350 days after this transaction.
This tells us the gap between this order and that customer's final recorded purchase.

Data Visualization

Preparing the Data: RFM Scores
Before move on to data visualization, i need to summarize all of a customer's activity into three key metrics known as **RFM**:

R (Recency): How **recently** the customer made a purchase.
F (Frequency): How **often** the customer makes purchases.
M (Monetary): How **much money** the customer has spent in total.

Why RFM is critical:
Identifying Churn: The **Recency** score is our main tool for defining which customers we consider "gone" or "at risk" (churned).
Prediction Feature: All three scores (R, F, and M) will be the most important features used when we build the predictive machine learning model.

2

Data Visualization

```
[49]: import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns

[50]: df = pd.read_csv("D:\GitHub Project\A_DATA ANALYST SIMULATION\DAY 2\customerorder.csv")
      df
```

[50]:

	CUST_ID	ORDER_ID	ORDER_DATETIME	DAYS_ELAPSED_TMAX	DAYS_FROM_CUST_LAST_ORDER	REVENUE	QUANTITY	STORE_ID	PROD
0	1	159	2021-04-20 12:51:57	357	271	39.16	2	1	
1	1	182	2021-04-26 03:56:02	351	265	41.32	4	1	
2	1	182	2021-04-26 03:56:02	351	265	56.42	2	1	

I'm now moving into the Jupyter Notebook environment for the heavy lifting, known as **Feature Engineering.**

Goals for the Final Dataset:
Visualization: The clean, final dataset will be used to build an interactive Dashboard in **Power BI.**
Modeling: The same dataset will be used to train and develop a robust **Machine Learning Predictive Model.**

I have identified **315 null values** in the **shipment_status** column, I will fill these missing values by assigning the label '**pending/norecord**' to ensure the data is complete before visualization and modeling.

```
[51]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   CUST_ID                   1000 non-null   int64
 1   ORDER_ID                  1000 non-null   int64
 2   ORDER_DATETIME            1000 non-null   object
 3   DAYS_ELAPSED_TMAX         1000 non-null   int64
 4   DAYS_FROM_CUST_LAST_ORDER 1000 non-null   int64
 5   REVENUE                   1000 non-null   float64
 6   QUANTITY                  1000 non-null   int64
 7   STORE_ID                  1000 non-null   int64
 8   PRODUCT_ID                1000 non-null   int64
 9   ORDER_STATUS              1000 non-null   object
 10  SHIPMENT_STATUS           685 non-null    object
dtypes: float64(1), int64(7), object(3)
memory usage: 86.1+ KB

[52]: df.isnull().sum()

[52]: CUST_ID                     0
      ORDER_ID                    0
      ORDER_DATETIME              0
      DAYS_ELAPSED_TMAX           0
      DAYS_FROM_CUST_LAST_ORDER   0
      REVENUE                     0
      QUANTITY                    0
      STORE_ID                    0
      PRODUCT_ID                  0
      ORDER_STATUS                0
      SHIPMENT_STATUS           315
      dtype: int64
```

Data Visualization

```
[53]: df['ORD_DATETIME'] = pd.to_datetime(df['ORDER_DATETIME'])

[54]: # handling null values with new variable Pending or no record
      df['SHIPMENT_STATUS'] = df['SHIPMENT_STATUS'].fillna('PENDING/NORECORD')

[55]: df['SHIPMENT_STATUS'].value_counts()

[55]: SHIPMENT_STATUS
      DELIVERED           685
      PENDING/NORECORD    315
      Name: count, dtype: int64
```

The shipment_status column is now complete, with all missing values successfully replaced by 'Pending/NoRecord'.

Data Visualization

```
[56]: # Define the aggregation for core features
agg_dict = {
    # Recency (R) & Tenure
    # R is the MINIMUM DAYS_ELAPSED_TMAX (since the smallest gap means the latest order)
    'DAYS_ELAPSED_TMAX': ['min', 'max'],

    # Monetary (M)
    'REVENUE': 'sum',

    # Frequency (F) - Count of unique order IDs
    'ORDER_ID': 'nunique',

    # 4. Item/Product Variety
    'PRODUCT_ID': 'nunique',
    'STORE_ID': 'nunique',

    # 5. IPI (Inter-Purchase Interval) - Only consider non-zero time differences
    'DAYS_FROM_CUST_LAST_ORDER': lambda x: np.mean(x[x > 0]),
    'QUANTITY': ['sum', 'mean']
}

customer_df = df.groupby('CUST_ID').agg(agg_dict).reset_index()
```

```
[57]: customer_df
```

[57]:

	CUST_ID	DAYS_ELAPSED_TMAX		REVENUE	ORDER_ID	PRODUCT_ID	STORE_ID	DAYS_FROM_CUST_LAST_ORDER	QUANTITY	
		min	max	sum	nunique	nunique	nunique	<lambda>	sum	mean
0	1	86	357	397.22	5	8	1	171.625000	22	2.444444
1	2	215	425	332.99	2	4	2	210.000000	17	4.250000
2	3	82	432	1765.44	10	19	2	231.550000	76	3.304348

```
[58]: # Flatten the multi-level column names
customer_df.columns = [
    'CUST_ID',
    'RECENCY_DAYS', 'TENURE_DAYS',
    'MONETARY', 'FREQUENCY_ORDERS',
    'PRODUCT_VARIETY', 'STORE_VARIETY',
    'IPI_AVG_DAYS',
    'TOTAL_QUANTITY', 'AVG_QUANTITY_PER_ITEM'
]
```

```
[59]: customer_df.head()
```

[59]:

	CUST_ID	RECENCY_DAYS	TENURE_DAYS	MONETARY	FREQUENCY_ORDERS	PRODUCT_VARIETY	STORE_VARIETY	IPI_AVG_DAYS	TOTAL_Q
0	1	86	357	397.22	5	8	1	171.625000	
1	2	215	425	332.99	2	4	2	210.000000	
2	3	82	432	1765.44	10	19	2	231.550000	
3	4	89	284	1263.27	7	12	2	130.833333	
4	5	224	320	150.74	3	3	2	92.500000	

Also define a new features name as you can see above.

Here i'm going to define the essential **aggregations** using a dictionary. This process groups all transactions by **Customer_id** to calculate the core RFM metrics and other necessary features:

Recency and Tenure: Calculated using the minimum and maximum values of days_elapsed_tmax.

Monetary: Calculated by summing the total Revenue.

Inter-Purchase Interval (IPI): A crucial metric that calculates the average time between a customer's purchases.

Data Visualization

```
•[60]: # Derived and Proportion Features
       # Average Order Value (AOV)
       customer_df['AOV'] = customer_df['MONETARY'] / customer_df['FREQUENCY_ORDERS']

       # Target Variable (CHURNED)
       # Apply rule: Churn if Recency (days since last purchase) > 90
       customer_df['CHURN'] = (customer_df['RECENCY_DAYS'] > 90).astype(int)

       # Service Status Proportions (Requires re-aggregation)
       status_df = df.groupby('CUST_ID')['SHIPMENT_STATUS'].value_counts(normalize=True).unstack(fill_value=0)
       status_df.columns = ['SHIP_' + col for col in status_df.columns]
```

```
[61]: customer_df.head()
```

[61]:

	CUST_ID	RECENCY_DAYS	TENURE_DAYS	MONETARY	FREQUENCY_ORDERS	PRODUCT_VARIETY	STORE_VARIETY	IPI_AVG_DAYS	TO
0	1	86	357	397.22	5	8	1	171.625000	
1	2	215	425	332.99	2	4	2	210.000000	
2	3	82	432	1765.44	10	19	2	231.550000	
3	4	89	284	1263.27	7	12	2	130.833333	
4	5	224	320	150.74	3	3	2	92.500000	

```
[62]: status_df.head()
```

[62]:

CUST_ID	SHIP_DELIVERED	SHIP_PENDING/NORECORD
1	1.000000	0.000000

Here i'm calculating the last set of critical predictive features:

Average Order Value (AOV): I establish the AOV by dividing the customer's total spending (Monetary) by the total number of orders they placed (Frequency_orders).

Defining Churn (The Target Variable): The most crucial step is creating the binary target variable, **Churn**. We are operationally defining a customer as "**Churned**" (or lost) if their calculated Recency value is greater than 90 days. This means any customer who has not transacted for over three months is flagged as having left the company.

Shipment Status Percentages: Finally, i calculate the relative percentages of 'Complete' and 'Pending/NoRecord' shipment statuses for each customer, providing a normalized view of their order fulfillment history.

Data Visualization

```
[63]: final_df1 = customer_df.merge(status_df, on='CUST_ID', how='left')
```

```
[64]: final_df1
```

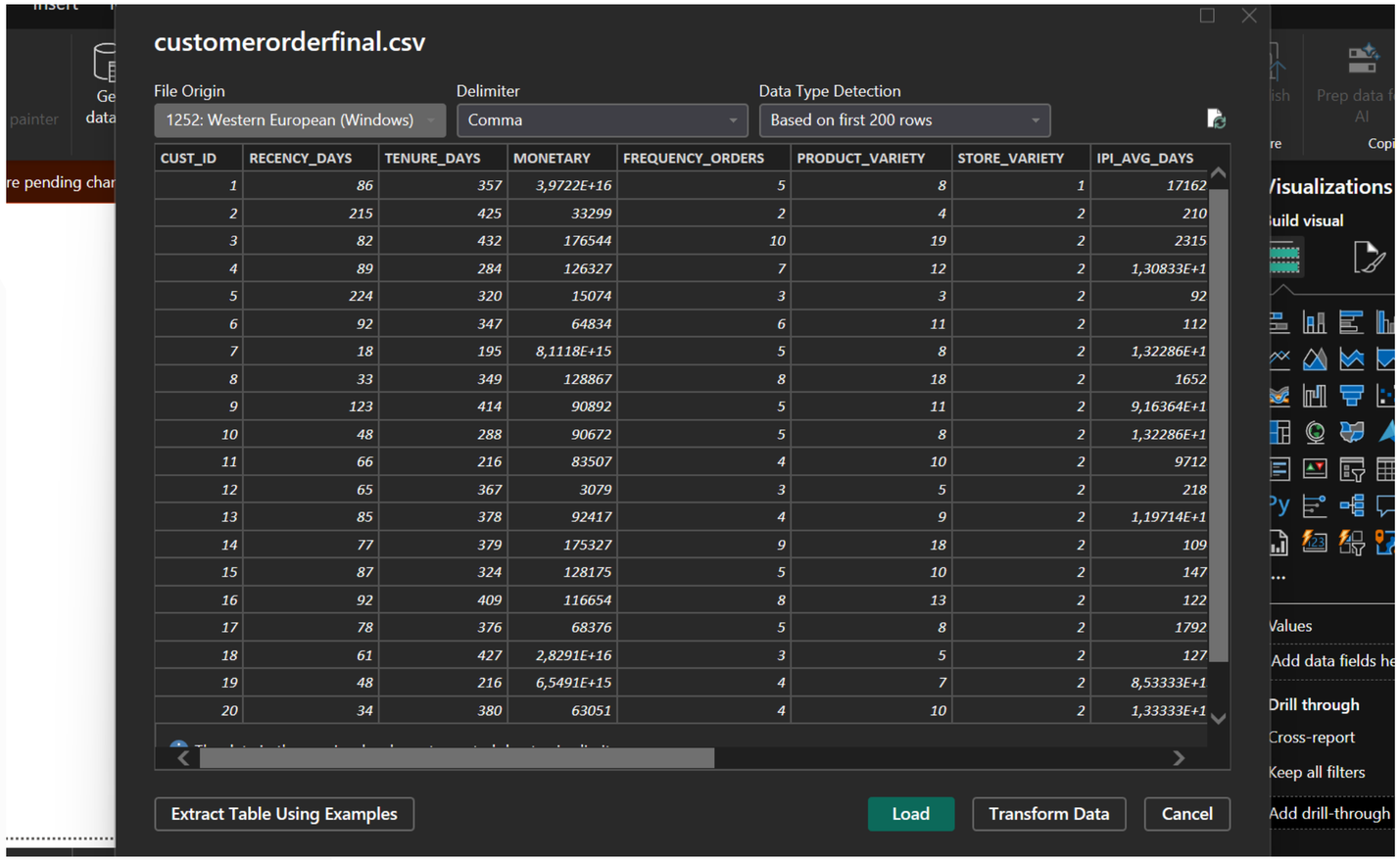
[64]:

	CUST_ID	RECENCY_DAYS	TENURE_DAYS	MONETARY	FREQUENCY_ORDERS	PRODUCT_VARIETY	STO
0	1	86	357	397.22	5	8	
1	2	215	425	332.99	2	4	
2	3	82	432	1765.44	10	19	
3	4	89	284	1263.27	7	12	
4	5	224	320	150.74	3	3	
...	...	...	...	...	...	...	
101	102	76	325	830.47	7	10	
102	103	62	221	392.52	4	6	
103	104	57	329	1051.78	6	12	
104	105	60	343	815.38	6	10	
105	106	367	413	454.35	2	6	

106 rows × 14 columns

The final step is to merge the engineered status_df (containing the shipment status percentages) back into the main DataFrame for a unified, final dataset ready for both visualization and modeling.

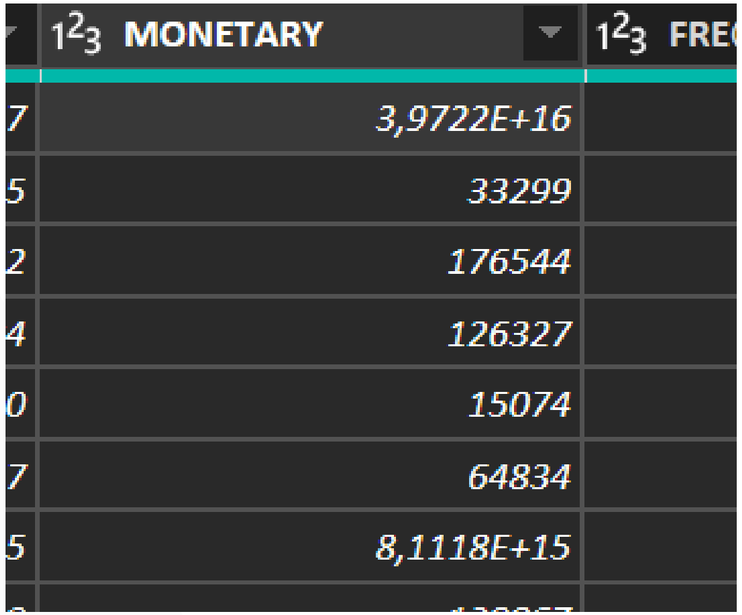
Data Visualization



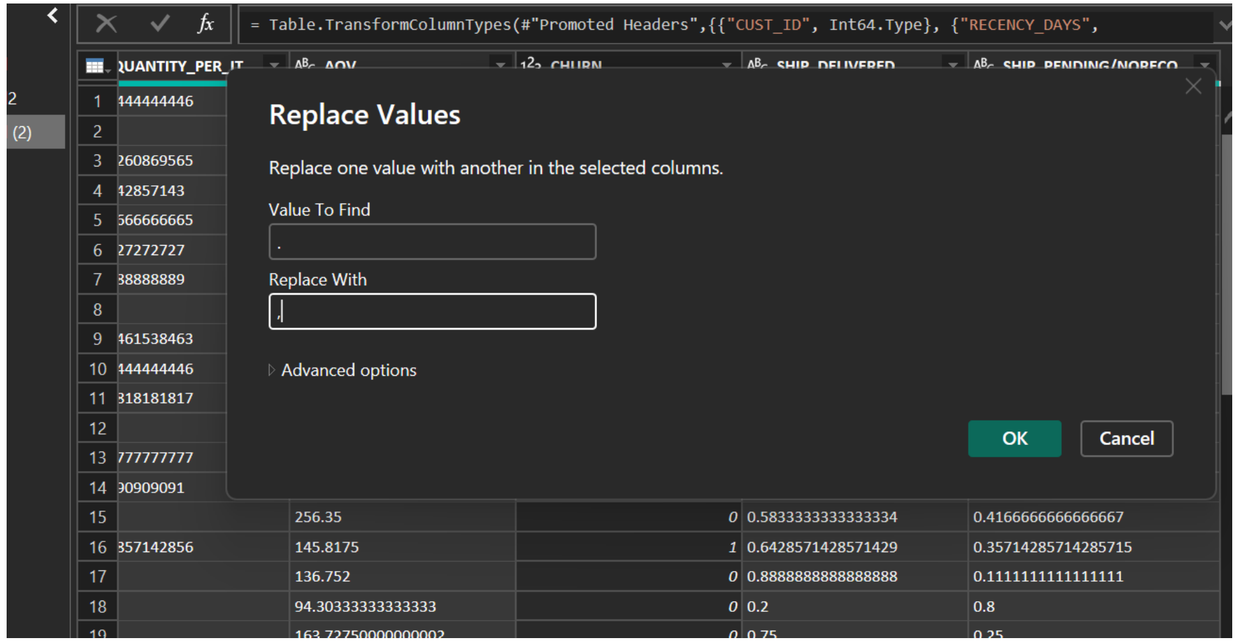
CUST_ID	RECENCY_DAYS	TENURE_DAYS	MONETARY	FREQUENCY_ORDERS	PRODUCT_VARIETY	STORE_VARIETY	IPI_AVG_DAYS
1	86	357	3,9722E+16	5	8	1	17162
2	215	425	33299	2	4	2	210
3	82	432	176544	10	19	2	2315
4	89	284	126327	7	12	2	1,30833E+1
5	224	320	15074	3	3	2	92
6	92	347	64834	6	11	2	112
7	18	195	8,1118E+15	5	8	2	1,32286E+1
8	33	349	128867	8	18	2	1652
9	123	414	90892	5	11	2	9,16364E+1
10	48	288	90672	5	8	2	1,32286E+1
11	66	216	83507	4	10	2	9712
12	65	367	3079	3	5	2	218
13	85	378	92417	4	9	2	1,19714E+1
14	77	379	175327	9	18	2	109
15	87	324	128175	5	10	2	147
16	92	409	116654	8	13	2	122
17	78	376	68376	5	8	2	1792
18	61	427	2,8291E+16	3	5	2	127
19	48	216	6,5491E+15	4	7	2	8,53333E+1
20	34	380	63051	4	10	2	1,33333E+1

Here I go with Power BI. I'm not jumping to load the data into my visualization, since I need some feature engineering here. Can you guess what's wrong?

Data Visualization

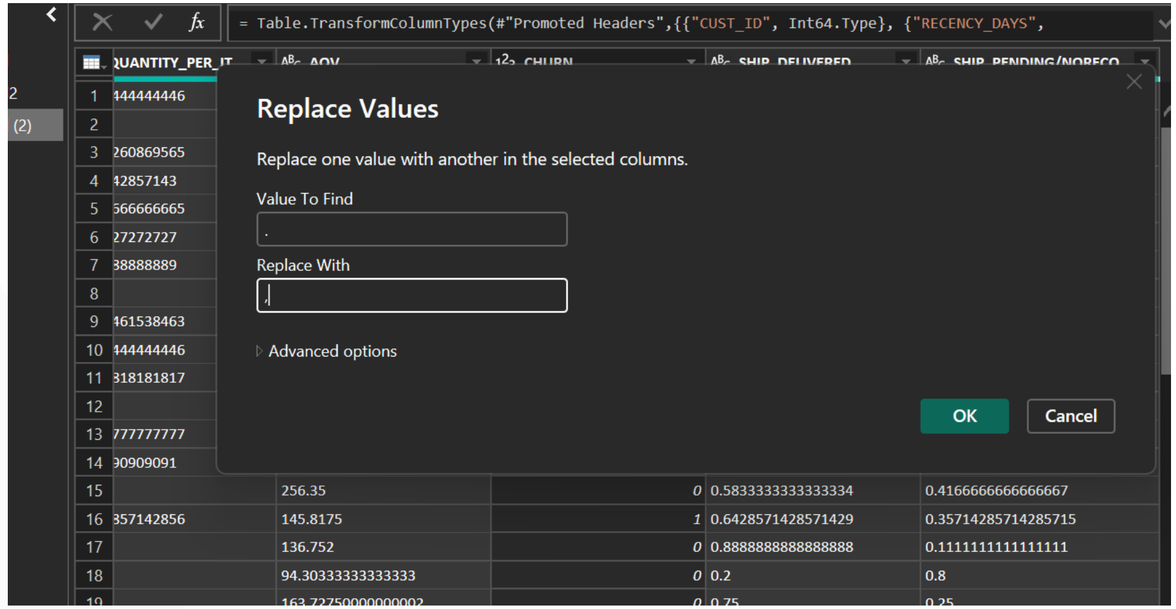


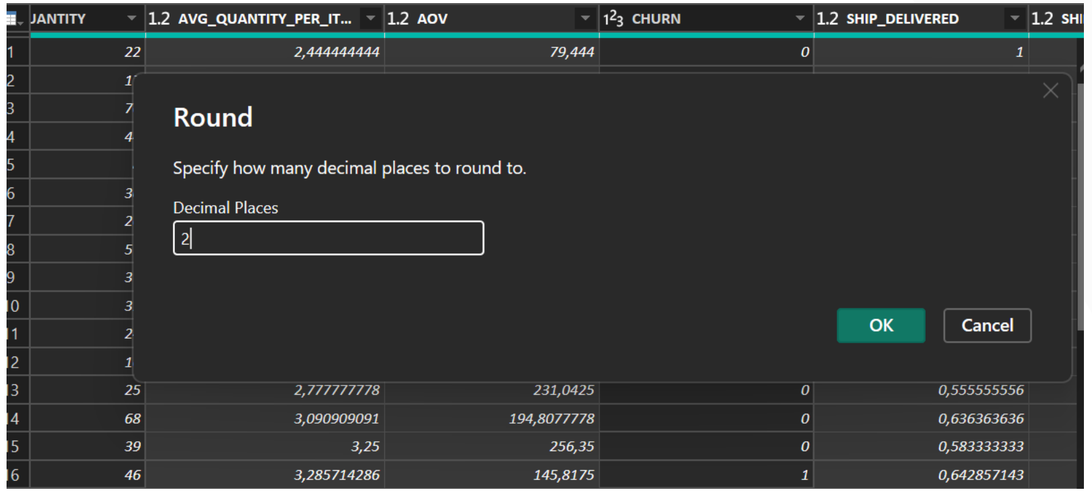
	MONETARY	FRE
7	3,9722E+16	
5	33299	
2	176544	
4	126327	
0	15074	
7	64834	
5	8,1118E+15	



As you can see, Power Query is different from Pandas. it reads the decimal number using a comma, not a dot. So, i need to replace the dot with a comma for Power Query to read it as a decimal number, don't forget to change the column type to Text first, and then back to Decimal after the replacement.

Data Visualization

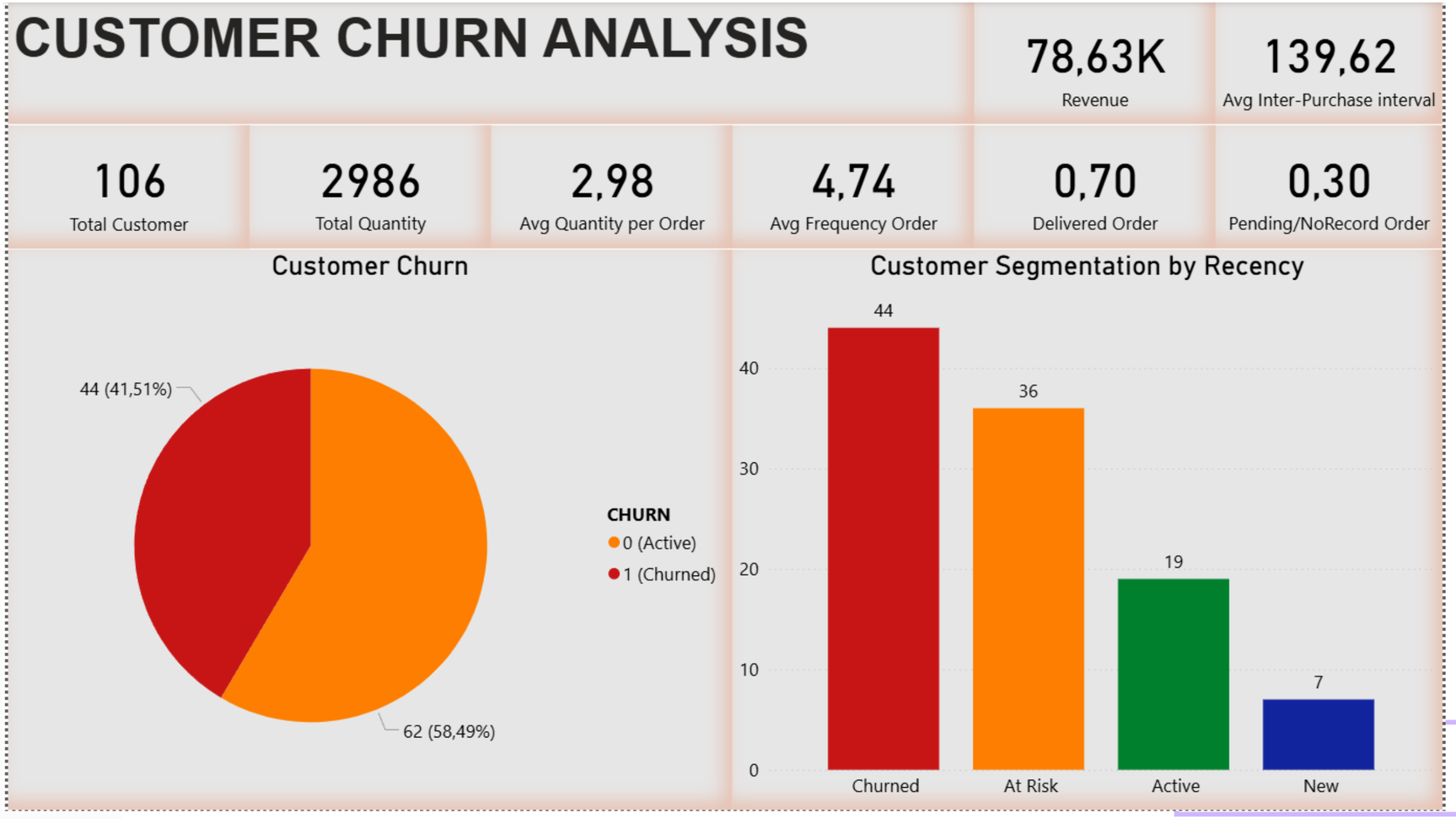




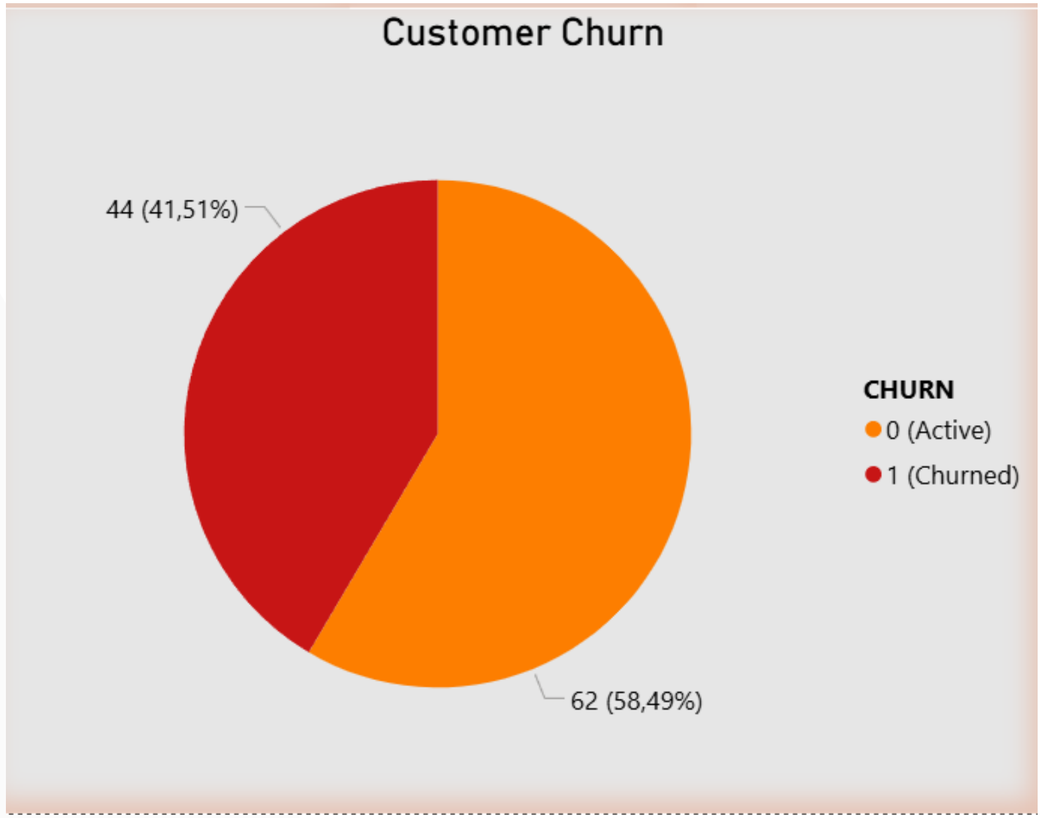
Additionally, round the values to two decimal places to ensure better readability and clearer presentation on the visualizations.

Data Visualization

Dashboard

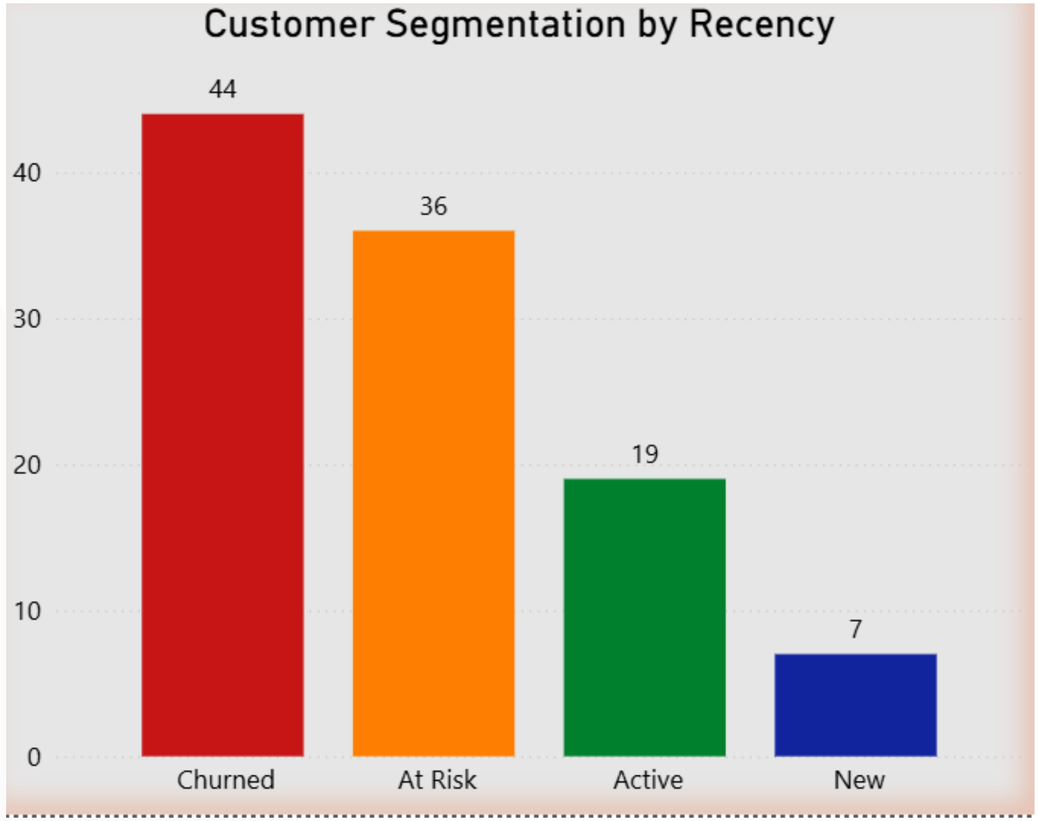


Data Visualization



The customer distribution is fairly balanced, with active customers (0) representing the majority at 58.49%, while the churned group (1) accounts for a significant 41.51%. This balanced ratio is also excellent for training a robust predictive model.

Data Visualization



Next, I created four distinct customer segments based on their **Recency** in days, following this distribution: **New** (Less than 21 days), **Active** (21-60 days), **At Risk** (61-90 days), and **Churned** (90+ days).

Out of the 106 customers, 41 are categorized as **Churned** and 36 are **At Risk** of churning, totaling 77 customers who require intervention. The remaining groups are smaller, with 19 **Active** customers and only 7 **New** customers. This distribution highlights a significant customer retention problem, as nearly three-quarters of our base is either gone or in danger of leaving.

Data Visualization

Conclusion

Conclusion on Customer Distributions:
The analysis of our customer base reveals two critical insights:

Churn Distribution: The overall binary split shows a significantly high churn rate, with 41.51% of customers classified as churned (1) against 58.49% active (0). This high proportion is beneficial for predictive modeling due to the balanced nature of the target variable but signals an urgent underlying issue with customer retention.

Segmentation Distribution: When segmented into four groups based on Recency (106 total customers), the retention challenge becomes even clearer: with **77 customers (41 Churned + 36 At Risk)** are either gone or highly vulnerable. Conversely, the healthy parts of the pipeline—Active (19 customers) and New (7 customers) are alarmingly small. This severely imbalanced segmentation confirms that customer churn is the primary issue demanding immediate strategic intervention.

Predictive Machine Learning Model

The final crucial step is to build a machine learning model to predict which customers are likely to churn. This predictive capability is essential for strategic intervention, allowing the company to proactively engage with at-risk customers and implement targeted retention efforts to minimize customer attrition and encourage renewed transactions.

To illustrate the advantage, a company may currently define a customer as 'churned' only after their **Recency exceeds 90 days** . However, the predictive model **eliminates the need to wait for this 90-day inactivity threshold**. Instead, we can forecast their likelihood of leaving based on their behavioral patterns when their recency is only 20-30 days, or even earlier. This capability allows the company to implement **intervention strategies much earlier**, well before the customer is definitively lost.

3

Predictive Machine Learning Model

Model Algorithm to use

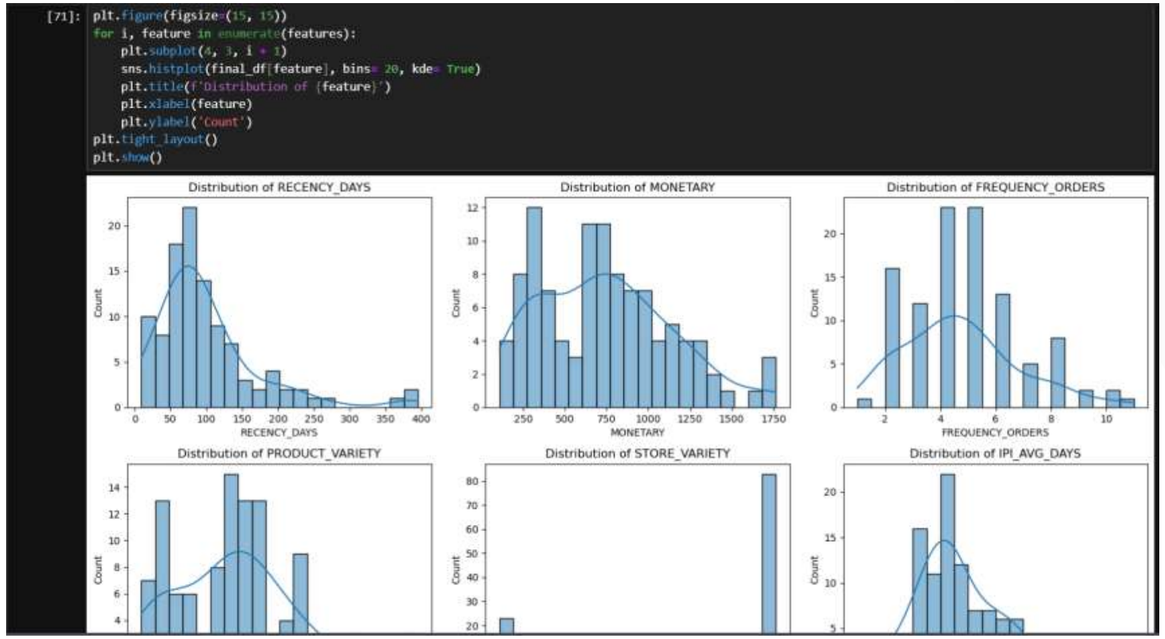
1. Logistic Regression
2. Decision Tree
3. K-Nearest Neighbors

Predictive Machine Learning Model

Exploratory Data Analysis (EDA)

```
[70]: plt.figure(figsize=(15, 15))
      for i, feature in enumerate(features):
          plt.subplot(4, 3, i + 1)
          sns.boxplot(x=churn_label, y=feature, data=final_df)
          plt.title(f'{feature} by Churn Status')
          plt.xlabel('Churn Status')
          plt.ylabel(feature)
      plt.tight_layout()
      plt.show()
```


```
[71]: plt.figure(figsize=(15, 15))
      for i, feature in enumerate(features):
          plt.subplot(4, 3, i + 1)
          sns.histplot(final_df[feature], bins= 20, kde= True)
          plt.title(f'Distribution of {feature}')
          plt.xlabel(feature)
          plt.ylabel('Count')
      plt.tight_layout()
      plt.show()
```



In this stage, I implemented visualization techniques to analyze the distribution of each feature and identify any outliers.
Crucially, for this specific model iteration, i opted **not** to perform any outlier treatment, choosing instead to maintain the purity and originality of the raw data distribution.

Predictive Machine Learning Model

Split Data into Train and Test Data

```
[24]: X = final_df[['MONETARY', 'SHIP_PENDING/NORECORD', 'IPI_AVG_DAYS', 'AVG_QUANTITY_PER_ITEM', 'TOTAL_QUANTITY']]
      y = final_df['CHURN']
      # the reason why we excluding the recency days because we define the Churn based on Recency value (>90 = Churn)
      # our model would get data leakage so its better to exclude Recency_days

[25]: from sklearn.model_selection import train_test_split, GridSearchCV
      from sklearn.preprocessing import StandardScaler
      from sklearn.linear_model import LogisticRegression
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, recall_score, precision_score, f1_score, roc_auc_score

[26]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

[27]: print(f'Training Size: {X_train.shape}')
      print(f'Test Size : {X_test.shape}')

      Training Size: (84, 5)
      Test Size : (22, 5)

[28]: X_test.isnull().sum()
```

Next, I divide the dataset into training and testing sets using a **0.2 split ratio**, resulting in **80% (84 samples)** allocated to the training data and **20% (22 samples)** allocated to the testing data.

Predictive Machine Learning Model

Standard Scaler & Defining Parameters

[77]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
```

Define Paramater for each Model

[78]:
```
lr_parameters = {
    'C': [0.01, 0.1, 1, 10, 100, 1000],
    'penalty': ['l2'],
    'solver': ['liblinear', 'lbfgs'],
    'class_weight': ['balanced', None]
}

dt_parameters = {
    'max_depth': [3, 5, 7, 10, 15, 20, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy'],
    'class_weight': ['balanced', None]
}

knn_parameters = {
    'n_neighbors': [1, 3, 5, 9, 15, 25],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
```

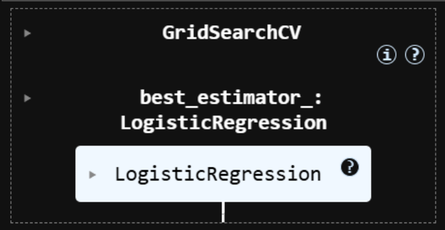
Next, i applied the **Standard Scaler** to standardize the features necessary step, particularly for **K-Nearest Neighbors (KNN)**, which relies on distance calculations and performs poorly when features have disparate scales. Subsequently, I perform **Hyperparameter Tuning** for each model to identify the optimal configuration that maximizes performance and ensures our models are precisely calibrated to the specific patterns in our small dataset.

Predictive Machine Learning Model

Logistic Regression

```
[79]: lr = LogisticRegression(random_state=42)
      lrcv = GridSearchCV(lr, lr_parameters, cv=20, scoring='roc_auc', n_jobs=-1)
      lrcv.fit(X_train_scaled, y_train)
```

[79]:




```
[80]: print(f'Best Hyper Parameters : {lrcv.best_params_}')
      print(f'Accuracy Score : {lrcv.best_score_}')
```

```
Best Hyper Parameters : {'C': 0.01, 'class_weight': 'balanced', 'penalty': 'l2', 'solver': 'liblinear'}
Accuracy Score : 0.6916666666666667
```

```
[81]: accuracy = lrcv.score(X_test_scaled, y_test)
      print(f'Logistic Regression Accuracy score : {accuracy}')
```

```
Logistic Regression Accuracy score : 0.8547008547008547
```

```
[82]: y_pred = lrcv.best_estimator_.predict(X_test_scaled)
      y_prob = lrcv.best_estimator_.predict_proba(X_test_scaled)[:, 1]
```

```
[83]: # Confusion Matrix
      print("\nConfusion Matrix:")
      print(confusion_matrix(y_test, y_pred))

      # Classification Report
      print("\nClassification Report:")
      print(classification_report(y_test, y_pred, target_names=['Active (0)', 'Churned (1)']))

      # ROC AUC Score
      roc_auc = roc_auc_score(y_test, y_prob)
      print(f"\nROC AUC Score: {roc_auc:.4f}")
```

```
Confusion Matrix:
[[10  3]
 [ 3  6]]

Classification Report:
              precision    recall  f1-score   support

  Active (0)       0.77      0.77      0.77        13
 Churned (1)       0.67      0.67      0.67         9

    accuracy                           0.73        22
   macro avg       0.72      0.72      0.72        22
weighted avg       0.73      0.73      0.73        22


ROC AUC Score: 0.8547
```

Here, I implemented **Grid Search** using a predetermined parameter space to systematically identify the **optimal hyperparameters** for each model, maximizing their predictive performance.

Predictive Machine Learning Model

Logistic Regression Conclusion

The optimized Logistic Regression model demonstrates a good and balanced performance profile, suitable for use as an initial churn predictor baseline.

Confusion Matrix Breakdown:

- **10 True Negatives (TN):** The model correctly predicted 10 active customers.
- **6 True Positives (TP):** The model correctly predicted 6 churned customers.
- **3 False Positives (FP):** The model incorrectly predicted 3 active customers as churners (Type I error).
- **3 False Negatives (FN):** The model incorrectly predicted 3 churned customers as active (Type II error).

The optimized Logistic Regression model's performance is **moderate** (73% accuracy) and **slightly skewed** toward the **Active (0)** class:

- **Overall Predictive Power (Test Accuracy: 0.73):** The overall ability to correctly classify new customers is **73%**. While still better than random, this is a moderate score, suggesting room for improvement. The original ROC AUC score (0.85) remains an important indicator of good discriminatory power, but the lower accuracy score (0.73) highlights that the chosen decision threshold might not be optimal.

Predictive Machine Learning Model

Logistic Regression Conclusion

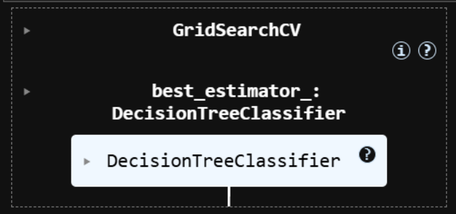
- **Imbalanced Class Performance:** The model performs better at identifying **Active** customers (Precision: 0.77, Recall: 0.77) than **Churned** customers (Precision: 0.67, Recall: 0.67).
- **Low Recall for Churn (67%):** The model fails to flag one-third (33%) of the truly churned customers (False Negatives). Since identifying churn risk is the goal, this moderate recall rate for the Churned class (1) is the most critical area for future model improvement.
- **Precision for Churn (67%):** When the model predicts churn, it is correct about two-thirds of the time. This means the remaining one-third of churn predictions are wasted efforts targeting customers who were never going to churn (False Positives).
- **Conclusion:** The model provides a reliable baseline for churn prediction with **73% accuracy**. However, its tendency to miss truly churned customers (Recall: 0.67 for Churn) means it may not be suitable for immediate deployment without further optimization focused on increasing the **Recall** for the **Churned (1)** class.

Predictive Machine Learning Model

Decision Tree

```
[148]: dt = DecisionTreeClassifier(random_state=42)
       dtcv = GridSearchCV(dt, dt_parameters, cv=10, scoring='roc_auc', n_jobs=-1)
       dtcv.fit(X_train_scaled, y_train)
```

[148]:




```
[134]: print(f'Best Hyper Parameters : {dtcv.best_params_}')
       print(f'Accuracy Score : {dtcv.best_score_}')
```

```
Best Hyper Parameters : {'class_weight': None, 'criterion': 'entropy', 'max_depth': 7, 'min_samples_leaf': 4, 'min_samples_split': 2}
Accuracy Score : 0.5916666666666666
```

```
[149]: accuracy2 = dtcv.score(X_test_scaled, y_test)
       print(f'Decision Tree Accuracy score : {accuracy2}')
```

```
Decision Tree Accuracy score : 0.7735042735042734
```

```
[150]: y_pred2 = dtcv.best_estimator_.predict(X_test_scaled)
       y_prob2= dtcv.best_estimator_.predict_proba(X_test_scaled)[:, 1]
```

```
[151]: # Confusion Matrix
       print("\nConfusion Matrix:")
       print(confusion_matrix(y_test, y_pred2))

       # Classification Report
       print("\nClassification Report:")
       print(classification_report(y_test, y_pred2, target_names=['Active (0)', 'Churned (1)']))

       # ROC AUC Score
       roc_auc = roc_auc_score(y_test, y_prob2)
       print(f"\nROC AUC Score: {roc_auc:.4f}")
```

```
Confusion Matrix:
[[10  3]
 [ 2  7]]

Classification Report:
              precision    recall  f1-score   support

  Active (0)       0.83      0.77      0.80        13
 Churned (1)       0.70      0.78      0.74         9

    accuracy                           0.77        22
   macro avg       0.77      0.77      0.77        22
weighted avg       0.78      0.77      0.77        22


ROC AUC Score: 0.7735
```

Predictive Machine Learning Model

Decision Tree Conclusion

The optimized Decision Tree model demonstrates strong and balanced predictive ability, achieving performance comparable to the previous baseline but offering a more favorable trade-off for business intervention.

Confusion Matrix Breakdown:

- **10 True Negatives (TN):** The model correctly predicted 10 active customers.
- **7 True Positives (TP):** The model correctly predicted 7 churned customers.
- **3 False Positives (FP):** The model incorrectly predicted 3 active customers as churners (Type I error).
- **2 False Negatives (FN):** The model incorrectly predicted 2 churned customers as active (Type II error).

The Decision Tree model is a solid performer with a 77% accuracy and a 0.7735 ROC AUC score. Its key strength lies in its balanced performance and low risk of missing true churners:

- **Strong Predictive Power:** The metrics confirm the model's reliability in separating the classes on new data. The consistency between Accuracy and ROC AUC suggests good generalization.

Predictive Machine Learning Model

Decision Tree Conclusion

- **High Recall for Churn (0.78):** This is the most critical improvement over the previous model. The Decision Tree correctly identifies 78% of all truly churned customers. Critically, it results in only 2 False Negatives out of 9 truly churned customers, meaning fewer high-risk customers slip through without intervention.

- **Acceptable Precision for Churn (0.70):** While the Precision for Churn is lower than the Active class, a score of 70% means that when the model recommends an intervention, it is correct 7 out of 10 times.

- **Actionable Rules:** The Decision Tree is inherently valuable because it allows for the extraction of explicit, easy to understand rules. This makes the model highly interpretable and actionable for business teams, regardless of its small edge in metrics.

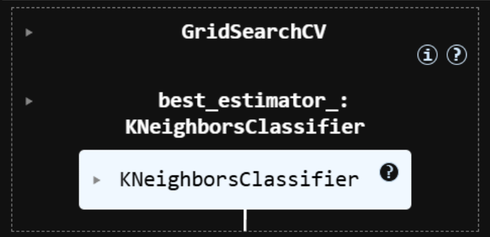
- **Conclusion:** The Decision Tree model provides an excellent and interpretable baseline for churn prediction. Its high **Recall (0.78)** for the churned class and low False Negative count make it particularly suitable for deployment where the cost of missing a churner is higher than the cost of a wasted intervention.

Predictive Machine Learning Model

K-Nearest Neighbors

```
[155]: knn = KNeighborsClassifier()
       knncv = GridSearchCV(knn, knn_parameters, cv=20, scoring='roc_auc', n_jobs=-1)
       knncv.fit(X_train_scaled, y_train)
```

[155]:




```
[139]: print(f'Best Hyper Parameters : {knncv.best_params_}')
       print(f'Accuracy Score : {knncv.best_score_}')
```

```
Best Hyper Parameters : {'metric': 'euclidean', 'n_neighbors': 25, 'weights': 'uniform'}
Accuracy Score : 0.6208333333333333
```

```
[140]: accuracy3 = knncv.score(X_test_scaled, y_test)
       print(f'KNN Accuracy score : {accuracy3}')
```

```
KNN Accuracy score : 0.8461538461538461
```

```
[160]: y_pred3 = knncv.best_estimator_.predict(X_test_scaled)
       y_prob3= knncv.best_estimator_.predict_proba(X_test_scaled)[:, 1]
```

```
[161]: # Confusion Matrix
       print("\nConfusion Matrix:")
       print(confusion_matrix(y_test, y_pred3))

       # Classification Report
       print("\nClassification Report:")
       print(classification_report(y_test, y_pred3, target_names=['Active (0)', 'Churned (1)']))

       # ROC AUC Score
       roc_auc = roc_auc_score(y_test, y_prob3)
       print(f"\nROC AUC Score: {roc_auc:.4f}")
```

```
Confusion Matrix:
[[12  1]
 [ 3  6]]

Classification Report:
              precision    recall  f1-score   support

  Active (0)       0.80      0.92      0.86        13
 Churned (1)       0.86      0.67      0.75         9

    accuracy                           0.82        22
   macro avg       0.83      0.79      0.80        22
weighted avg       0.82      0.82      0.81        22


ROC AUC Score: 0.8462
```

Predictive Machine Learning Model

K-Nearest Neighbors Conclusion

The optimized KNN model delivered the **highest overall accuracy** and the **best precision** among the three models, establishing it as the top performer for efficiently targeting active customers.

Confusion Matrix Breakdown:

- **12 True Negatives (TN):** The model correctly predicted 12 active customers.
- **6 True Positives (TP):** The model correctly predicted 6 churned customers.
- **1 False Positives (FP):** The model incorrectly predicted 1 active customers as churners (Type I error).
- **3 False Negatives (FN):** The model incorrectly predicted 2 churned customers as active (Type II error).

The KNN model demonstrates **excellent overall predictive power** (Accuracy: 0.82, ROC AUC: 0.8462), but its strengths are clearly concentrated on the Active (0) class and achieving high precision:

- **High Efficiency and Precision (Churn Precision: 0.86):** The model excels at minimizing false alarms. When the KNN model predicts a customer will churn, it is correct **86%** of the time. This is its most valuable asset, as the low **False Positive** count (only 1) ensures that retention efforts are highly efficient and not wasted on customers who were likely to stay anyway.

Predictive Machine Learning Model

K-Nearest Neighbors Conclusion

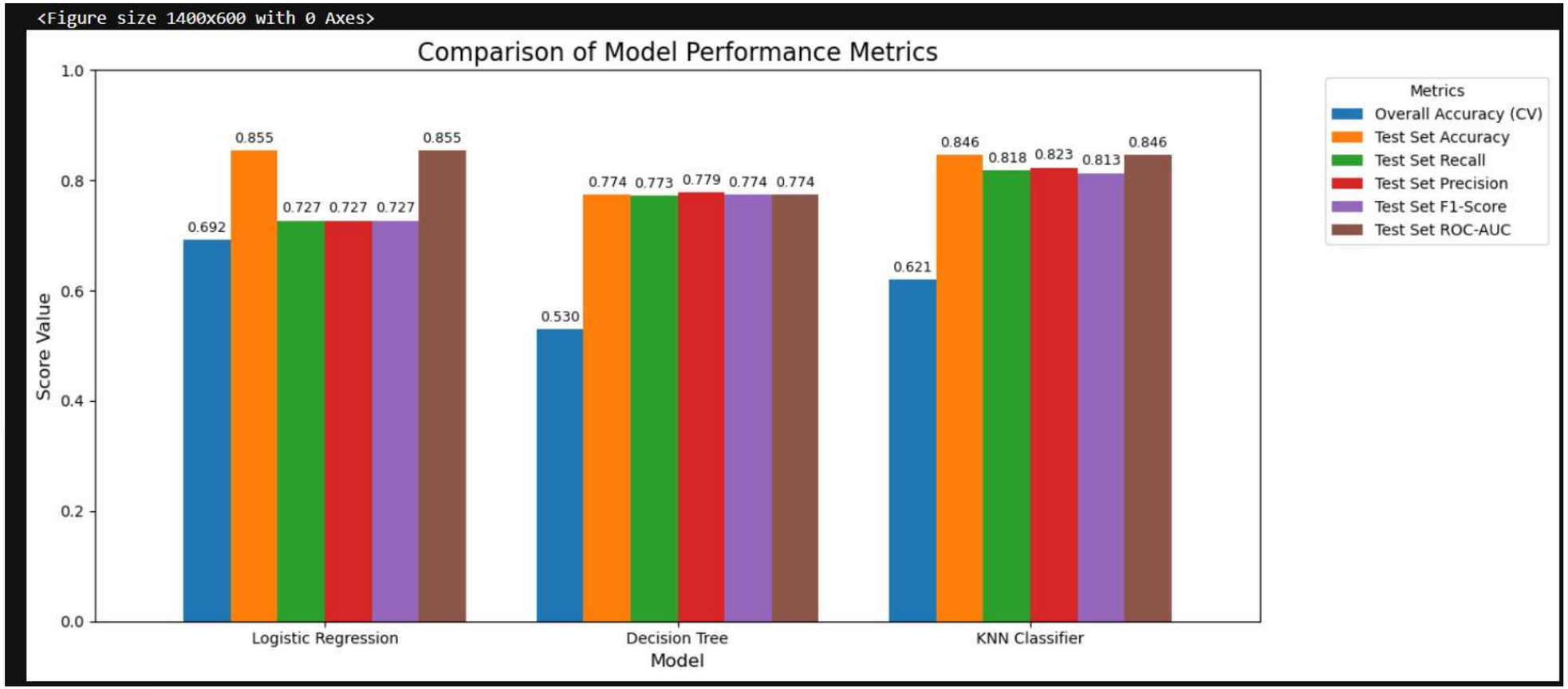
- **Excellent Active Recall (Recall: 0.92):** The model correctly identifies **92%** of all truly active customers, demonstrating its robust ability to confirm safe customers.

- **Moderate Churn Recall (Recall: 0.67):** The trade-off for high precision is a lower recall for the churned class. The model missed **3** of the truly churned customers (33% of churners were missed). While the Decision Tree had a better churn recall (0.78), the KNN's superior precision makes it a compelling choice.

- **Conclusion:** The KNN model is a highly precise and reliable predictor with the best overall accuracy and ROC AUC score. It is the ideal candidate for budget-conscious companies where minimizing wasted intervention costs (False Positives) is the primary business objective. However, teams must be aware that this precision comes at the cost of missing more true churners than the Decision Tree model.

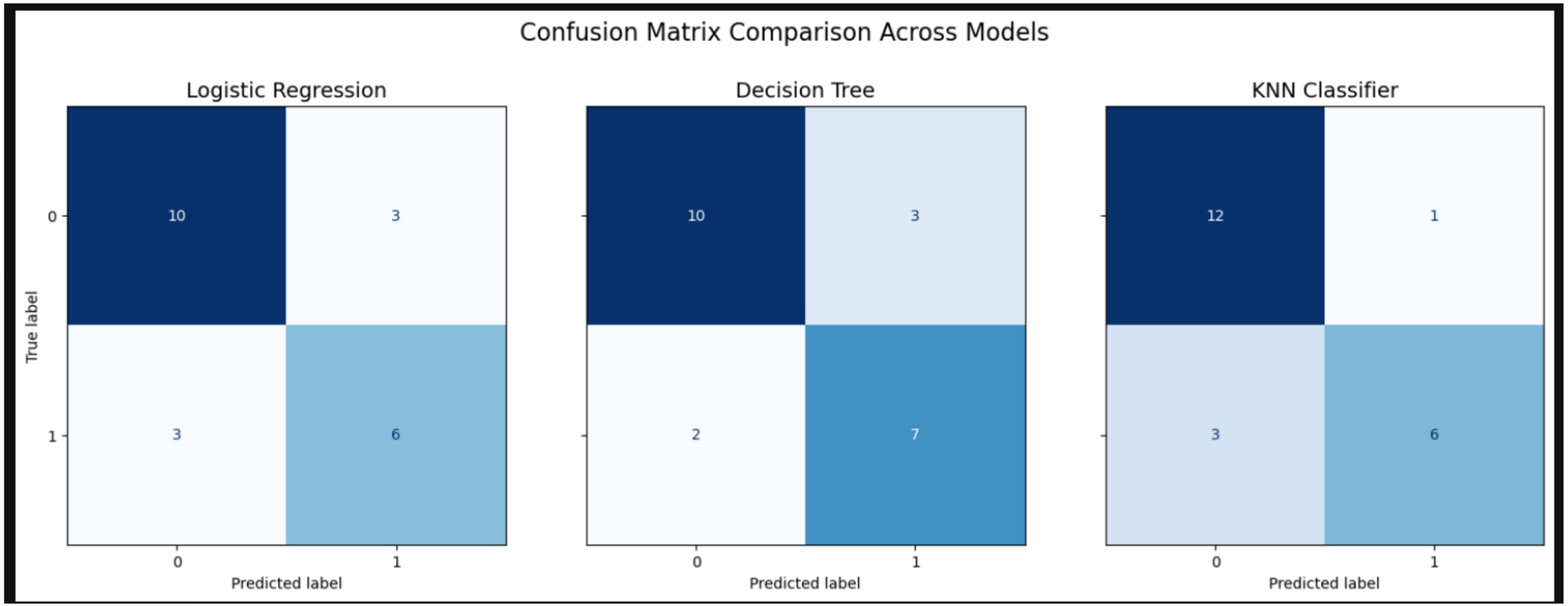
Predictive Machine Learning Model

Overall Model Performance



Predictive Machine Learning Model

Overall Model Performance



Predictive Machine Learning Model

Final Conclusion

Based on the detailed Classification Reports across all three models, the **K-Nearest Neighbors (KNN)** model is the **superior choice** for initial deployment, as it provides the **highest overall accuracy** and the most precise classification of the critical **Churned (1) class**.

Superior Efficiency and Reliability (Precision & F1-Score)

- The KNN model excels where budget efficiency and trust in a prediction are paramount:
- **Highest Targeting Precision:** When the KNN model flags a customer for an intervention, it is correct 86% of the time. This is its most significant advantage, leading to the lowest False Positive count (1) across all models. For budget-conscious companies, this directly translates to minimal wasted spending on retention offers given to customers who were likely to stay anyway.

High Balanced Performance

- The F1-Score of 0.75 confirms a much better harmony between correctly identifying churners and avoiding false flags compared to the Logistic Regression (0.67). Even compared to the Decision Tree (F1 Churn 0.74), the KNN is marginally stronger, making its predictions highly reliable for actionable decision-making.

Predictive Machine Learning Model

Final Conclusion

Highest Overall Generalization

- KNN delivered the highest overall scores, demonstrating the strongest performance on unseen data:
- **Highest Test Accuracy: 0.82**
- **Highest ROC AUC Score: 0.8462**

Addressing the Trade-Off (Recall vs. Precision)

- While the Decision Tree achieved a higher Churn Recall (0.78 vs. KNN's 0.67) by missing only 2 churners (vs. KNN's 3), choosing KNN signifies a strategic decision:

KNN prioritizes the quality of the churn prediction (Precision) over the quantity (Recall).

- It minimizes the cost of **False Alarms (FP)**, accepting a slightly higher risk of **Lost Revenue (FN)**.
- This is the correct choice when retention resources are expensive, and the company needs high confidence that every targeted customer is genuinely at risk.

Conclusion: The KNN model is the most effective and efficient tool for immediate deployment. Its high precision and low False Positive count ensure that retention resources are utilized optimally, providing the best return on investment for churn mitigation efforts.

Thank You

For further information regarding this analysis or my experience, please feel free to reach out using the following channels:

Email : yusrifargfar@gmail.com
WhatsApp : +62 896 8511 3550

You can review my professional activities and projects on the platforms below:

LinkedIn : www.linkedin.com/in/yusrifar-gaffar-69bb38323
GitHub : www.github.com/yusrifargfar

Credits

This presentation template is free for everyone to use thanks to the following:

SlidesCarnival for the presentation template

Pexels for the photos

Happy designing!